Homework 7

1 Verifying Distances

1.1 a

Designing a *linear time* algorithm that takes as input hte adjacency list G as a nested hash table, source s, and hash table $dist$. Return boolean expressions to verify that all distances in $dist$ are valid shortest path from s to all other vertices after running Dijkstra's Algorithm.

1.1.1 Pseudocode

Algorithm 1: VerifyDijkstra(G, s,$dist$)

```
   /* G: input adjacency list as a nested hash table                          */
   /* s:  source vertex                                                       */
   /* dist:  distances from source vertex s to all other vertices(hash table) */
 1 if dist[s] ≠ 0 then
       /* if the source node's distance is not 0, invalid                    */
 2     return "incorrect"
   /* case 1:  if π(v) > d(u) + l(u,v)                                        */
   /* we should maintain the invariant for all edges in G for valid Dijkstra's output
      */
   /* It is absolutely not valid if π(v) > d(u) + l(u,v) for any edge because that means
      either there is a shorter path from s to v through u or there exists a negative
      weight edge in the graph.                                               */
 3 for u in G do
 4     for v in G[u] do
 5         weight ← G[u][v] if dist[v] > dist[u] + weight then
 6             return "incorrect"
   /* case 2:  after setting up upper bound for the candidates, we only need the ones
      that strictly satisfy the invariant from the class regarding the valid distance
      of each node                                                            */
   /* First, run BFS on G so that we check every node in G, just like Dijkstra's
      algorithm does for shortest path by visiting every node in the graph.   */
 7 opt_cand ← empty hash table Q ← empty queue /* this is not a Priority Queue so there
      is no extract_min() and decrease_key() operations that would require additional
      logrithmic runtime.                                                     */
 8 Q.enqueue(s) opt_cand.add(s) while !Q.isEmpty() do
 9     u ← Q.dequeue()
10     for v in G[u] do
11         weight ← G[u][v] /* we only add the ones that are valid by the invariant:
               pi(v) = d(u) + l(u,v)                                          */
12         if dist[u] = dist[v] + weight then
13             if v ∉ opt_cand then
14                 opt_cand.add(v) Q.enqueue(v)
   /* now we check if the possible valid candidate hash table is correct      */
   /* 1.  if the node is in the candidate hash table (or we included that node as one
      of the shortest pathways in BFS), then its weight should not be infinity because
      we recorded the constant weight                                         */
   /* 2.  otherwise, the weight should be infinity because we do not go through that
      node for shortest path.                                                 */
15 for v in G do
16     if v ∈ opt_cand then
17         if dist[v] = ∞ then
18             return "incorrect"
19     else
20         if dist[v] ≠ ∞ then
21             return "incorrect"
22 return "correct"
```

1.1.2 Runtime Analysis

1. $O(1)$: constant time to check if source node is correct
2. $O(n+m)$: linear time to set the upper bound for all nodes in the graph ($\pi(v) > min_{u \in S, v \in adj(u)} d(u) + l(u,v)$)
3. $O(n+m)$: linear time to run BFS to create possible candidate distance hash table that Dijkstra's algorithm would have created.
4. $O(n)$: linear time to compare the input distance table with possible candidate distance table to see if there exsits any node missing or overcounted.

Therefore, the total runtime of the algorithm above is $O(1)+O(n+m)+O(n+m)+O(n) = O(n+m)$.

1.1.3 Proof of Correctness

For Dijkstra's algorithm to be correct:

1. $\pi(v) = min_{u \in S, v \in adj(u)} d(u) + l(u,v)$
2. If $u \in S$, then $d(u) = \delta(s,u)$
3. Every vertex in G does not have weights of ∞ if it is reachable from the source vertex s.

The algorithm checks if those three claims hold for the distance hash table $dist$.

1. The first claim is checked in the first case ($\pi(v) > min_{u \in S, v \in adj(u)} d(u) + l(u,v)$) in the algorithm to restrict upper bound of $\pi(v)$ for each node in G to $\pi(v) = min_{u \in S, v \in adj(u)} d(u) + l(u,v)$.
2. Then, with the BFS implementation to iterate through G to verify the equality.
3. Lastly, to check if every node on the shortest path is accounted, we run the last for loop to check if the node is in the candidate hash table and the node's weight is not ∞ or vise versa.

Therefore, the algorithm always verify the distance produced by Dijkstra's algorithm that $dist$ is the shortest path.

1.2 (b) Parent Reconstruction Algorithm

1.2.1 Pseudocode

Algorithm 2: reconstruct(G, s,$dist$)

```
   /* G: input adjacency list as a nested hash table                          */
   /* s:  source vertex                                                       */
   /* dist:  distances from source vertex s to all other vertices(hash table) */
   /* the maximum length of the parent hash table is the number of vertices; space:
      O(n)                                                                    */
 1 parents ← hash table with n null values filled /* n = |V|                 */
   /* loop through the adjacency table to find parents of each node that goes through
      shortest path                                                           */
   /* dijkstra's algorithm determines the shortest path with greedy approach that
      selects a node of a distance that equals the sum of minimum total distance of
      previous node that directs to current node and the weight               */
 2 for u in G do
 3     for v in G[u] do
 4         weight ← G[u][v] /* weight extraction from G                       */
           /* Three conditions to check                                       */
           /* 1.  make sure it is not a source node; it is root such that it cannot
              have a parent                                                   */
           /* 2.  make sure we visited that node so dist[u] cannot be ∞       */
           /* 3.  distance of current node must be equal to the sum of minimum distance
              of previous node and its weights                                */
 5         if v != s ∩ dist[u] != ∞ ∩ dist[v] == dist[u] + weight then
 6             parents[v] ← u /* update parent if passed the conditions       */

 7 return parent
```

1.2.2 Runtime Analysis

- $O(n)$: 'parents' hash table initialization. Because we have to initialize it with at most length n, we need to iterate through n times to assign 'null' value to each index of 'parents'
- $O(n+m)$: we have to iterate through G adjacency graph to check the conditions given above. while condition check only takes constant time $O(1)$, we need to check for every vertex to find the parent nodes through the shortest path possible.

Therefore, the total runtime of the algorithm is $O(n) + O(n+m) = O(n+m)$.

1.2.3 Proof of Correctness

To reconstruct 'parents' hash table, a table that contains the parent of each node that goes through the shortest path,

- we have to make sure if 'dist' given is valid
- we have to make sure if the parent of the source node does not exist
- we have to make sure if all nodes in 'parents' were visited

- we have to make sure if the distance of child node must equal to the sum of the minimum distance of parent node and the edge weight between the child and its parent nodes.

Because the algorithm assumes that the output of (a) is "correct", 'dist' must always be valid and contains the shortest path from the source to every node in G. Also, 2,3, and 4 are checked at if condition inside the for loop that iterates through the adjacency list G.

Therefore, the algorithm above always reconstructs 'parents' hash table that has every edge between nodes through the shortest path correctly.

1.3 c

